Bloco · Algoritmos · Listas, pilhas e filas (10 questões · diagnóstico)

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.

> Prova diagnóstica, feita antes da teoria, para medir o que ficou da faculdade. Assunto único: listas, pilhas e filas. Marque "?" no que não souber e pontinho nas candidatas. Nunca deixe em branco.

Questão 1

O sistema e-PROTOCOLO guarda, em uma pilha, os documentos abertos pelo servidor, para permitir a função "desfazer". O analista Rogério testou o trecho em Python a seguir, em que a lista é usada como pilha (append empilha e pop desempilha):

```
pilha = []
for doc in ["P1", "P2", "P3"]:
    pilha.append(doc)
pilha.pop()
pilha.append("P4")
pilha.append("P5")
x = pilha.pop()
y = pilha.pop()
print(x, y, pilha)
```

Ao final da execução, será impresso:

(A) P2 P3 ['P4', 'P5'];

(B) P5 P4 ['P1', 'P2'];

(C) P4 P5 ['P1', 'P2'];

(D) P5 P4 ['P1', 'P2', 'P3'];

(E) P1 P2 ['P3', 'P4', 'P5'].

Questão 2

O sistema e-BALCÃO, do balcão virtual de um Tribunal de Justiça, atende as solicitações por ordem de chegada. Por determinação legal, porém, os pedidos de pessoas idosas ou com deficiência devem ser atendidos antes dos demais; entre os pedidos de mesma prioridade, continua valendo a ordem de chegada. A analista Carla precisa escolher a estrutura de dados que organiza a fila de atendimento.

A estrutura mais adequada é:

(A) pilha, pois o último pedido prioritário a chegar deve ser o primeiro a ser atendido;

(B) fila simples (FIFO), pois a ordem de chegada é o único critério de atendimento;

(C) deque, pois os pedidos podem ser inseridos e removidos por qualquer das extremidades, sem critério de ordem;

(D) fila de prioridade, em que sai primeiro o pedido de maior prioridade e, entre pedidos de mesma prioridade, o que chegou antes;

(E) lista circular, pois o último pedido aponta para o primeiro, o que garante que todos sejam atendidos.

Questão 3

No sistema e-PAUTA, escrito em Java, a operação mais frequente sobre a lista de audiências do dia é a inclusão de uma audiência urgente no **início** da lista. O acesso a uma audiência pela posição (índice) é raro. O analista Vítor avalia substituir o ArrayList atual por um LinkedList.

Sobre essa decisão, é correto afirmar que o analista deve escolher:

(A) LinkedList, pois a inserção no início é O(1), enquanto no ArrayList é O(n), já que todos os elementos precisam ser deslocados;

(B) ArrayList, pois a inserção no início de um vetor dinâmico é O(1), por causa da capacidade reservada;

(C) LinkedList, pois o acesso por índice na lista encadeada é O(1), o que também acelera a inserção;

(D) ArrayList, pois a LinkedList exige O(n) para inserir no início, já que precisa percorrer a lista até o último nó;

(E) qualquer das duas, pois ambas têm inserção no início em O(1) e acesso por índice em O(1).

Questão 4

O analista Gustavo escreveu a função a seguir para o validador de fórmulas do sistema e-CÁLCULO, que verifica se parênteses, colchetes e chaves de uma expressão estão balanceados:

```
def valida(expr):
    pares = {')': '(', ']': '[', '}': '{'}
    pilha = []
    for c in expr:
        if c in '([{':
            pilha.append(c)
        elif c in ')]}':
            if not pilha or pilha.pop() != pares[c]:
                return False
    return len(pilha) == 0
```

A chamada que retorna True é:

(A) valida("([)]");

(B) valida("{[()]}");

(C) valida("(()");

(D) valida("][()");

(E) valida("{[()()]}").

Questão 5

O sistema e-INTIMA usa uma fila circular, implementada em um vetor de 5 posições, para enfileirar as intimações a serem enviadas. Considere o código Python a seguir:

```
N = 5
fila = [None] * N
inicio, fim, qtd = 0, 0, 0

def enfileira(x):
    global fim, qtd
    fila[fim] = x
    fim = (fim + 1) % N
    qtd += 1

def desenfileira():
    global inicio, qtd
    x = fila[inicio]
    inicio = (inicio + 1) % N
    qtd -= 1
    return x
```

A analista Renata executou, nesta ordem: enfileira(10), enfileira(20), enfileira(30), enfileira(40), desenfileira(), desenfileira(), enfileira(50), enfileira(60) e enfileira(70).

Após essa sequência, os valores de inicio e de fim e o próximo valor a ser desenfileirado são, respectivamente:

(A) 2, 7 e 30;

(B) 0, 2 e 10;

(C) 2, 2 e 30;

(D) 2, 2 e 60;

(E) 4, 2 e 50.

Questão 6

O visualizador de autos do sistema JUSTIÇAPB+ tem os botões "Voltar" e "Avançar" para navegar entre as peças já abertas pelo magistrado, como em um navegador web. Quando uma nova peça é aberta, o histórico de "Avançar" é descartado. O analista Sérgio precisa implementar o recurso com inserção e remoção em tempo O(1).

A solução mais adequada é usar:

(A) uma única fila, em que "Voltar" remove do início e "Avançar" insere no fim;

(B) uma fila de prioridade, ordenada pela data de juntada de cada peça;

(C) duas pilhas, uma para "Voltar" e outra para "Avançar", esvaziando a pilha de "Avançar" quando uma nova peça é aberta;

(D) duas filas, uma para "Voltar" e outra para "Avançar", esvaziando a fila de "Voltar" quando uma nova peça é aberta;

(E) uma tabela hash, indexada pelo número da peça, que guarda a ordem de visita de cada peça.

Questão 7

A analista Beatriz revisa o método a seguir, do sistema SISGUIAS, escrito em Java, que remove o nó seguinte a um nó informado de uma lista simplesmente encadeada:

```
class No {
    int valor;
    No prox;
}

void removeApos(No ant) {
    if (ant != null && ant.prox != null) {
        ant.prox = ant.prox.prox;
    }
}
```

A lista contém, nesta ordem, os valores 3 → 8 → 5 → 9 → null, e a variável p aponta para o nó de valor 3. Em seguida, são executadas as chamadas removeApos(p.prox) e removeApos(p), nessa ordem.

Ao final, a lista será:

(A) 3 → 9 → null;

(B) 3 → 5 → 9 → null;

(C) 3 → 8 → 9 → null;

(D) 8 → 9 → null;

(E) 9 → null.

Questão 8

O sistema e-ARQUIVO mantém os processos arquivados em uma lista simplesmente encadeada com n nós. A implementação guarda apenas uma referência para o primeiro nó (head) e não guarda referência para o último nó. A lista não é mantida ordenada.

As complexidades de tempo das operações de inserir um processo no início da lista, inserir um processo no fim da lista e buscar um processo pelo número são, respectivamente:

(A) O(1), O(1) e O(1);

(B) O(n), O(1) e O(log n);

(C) O(1), O(1) e O(n);

(D) O(1), O(n) e O(log n);

(E) O(1), O(n) e O(n).

Questão 9

O analista Márcio revisa o trecho a seguir, em Python, do sistema e-REMESSA, que reorganiza a fila de lotes a serem enviados ao tribunal superior. Na lista, append empilha e pop desempilha:

```
from collections import deque

fila = deque([4, 7, 1, 9])
pilha = []
while fila:
    pilha.append(fila.popleft())
pilha.pop()
while pilha:
    fila.append(pilha.pop())
fila.append(fila.popleft())
print(list(fila))
```

Ao final da execução, será impresso:

(A) [1, 7, 4];

(B) [7, 4, 1];

(C) [1, 7, 9];

(D) [7, 1, 4];

(E) [1, 7, 4, 9].

Questão 10

O sistema e-DISTRIBUI processa, a cada madrugada, cerca de um milhão de processos em ordem de chegada. A analista Juliana comparou as duas versões a seguir, em Python:

```
# Versão 1
fila = []
for p in novos:
    fila.append(p)
while fila:
    distribui(fila.pop(0))

# Versão 2
from collections import deque
fila = deque()
for p in novos:
    fila.append(p)
while fila:
    distribui(fila.popleft())
```

A versão 1 mostrou-se muito mais lenta que a versão 2. A explicação correta é que:

(A) list.pop(0) é O(1), mas append em lista é O(n), o que torna lenta a montagem da fila na versão 1;

(B) as duas versões têm a mesma complexidade assintótica, e a diferença se deve apenas a constantes de implementação;

(C) deque é implementado como pilha, e por isso popleft() remove o último elemento inserido em O(1);

(D) list.pop(0) é O(n), pois desloca todos os elementos restantes, enquanto deque.popleft() é O(1);

(E) deque mantém os elementos ordenados, o que permite remover o menor elemento em O(log n).

Minhas respostas

Questão	Resposta
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	